LOGIFLOW

Smart Delivery Management System

Smart Logistics & Delivery Management

A Project Report Submitted in Partial Fulfillment of the Requirements for the Award of the Degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

Konda Vaishali - 2520030117

P. Bindhu Madhavi - 2520030077

Alaparthi Nava Harsha - 2520030493

Section 7 | Team 3

Database Systems Engineering and Distributed Backend Development (25CS1302E)
Trimester 4 | Academic Year 2026-2027

Under the Esteemed Guidance of

Dr. Spandana. S

Department of Computer Science and Engineering

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

The Project Report entitled “Smart Delivery Management System” is a record of bonafide work of KONDA VAISHALI - 2520030117, P. BINDHU MADHAVI - 2520030077 and ALAPARTHI NAVA HARSHA - 2520030493, submitted in partial fulfillment for the award of B.Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other department, university or institute.

Konda Vaishali - 2520030117

P. Bindhu Madhavi - 2520030077

Alaparthi Nava Harsha - 2520030493

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report entitled “Smart Delivery Management System”, submitted by Konda Vaishali (2520030117), P. Bindhu Madhavi (2520030077) and Alaparthi Nava Harsha (2520030493), is submitted for the course Database Systems Engineering and Distributed Backend Development (25CS1302E), during the academic year 2026-2027, Trimester 4.

The report has been prepared under the guidance of Dr. Spandana. S, Department of Computer Science and Engineering. Approval of the work and its acceptance for academic submission are to be confirmed by the authorised faculty signatures below.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the timely help, support and guidance provided by many people. We would like to express our sincere gratitude to everyone who assisted us directly or indirectly in completing our project, “Smart Delivery Management System.”

We express our deepest appreciation to our Course Coordinator and Project Guide, Dr. Spandana. S, Department of Computer Science and Engineering, for her valuable guidance, continuous support, encouragement and constructive suggestions throughout the development of this project.

We express our gratitude to the Head of the Department, Department of Computer Science and Engineering, for providing the facilities, resources and support required to complete this project-based work.

We also thank all the teaching and non-teaching staff members of the department for their assistance and cooperation, and finally our parents, friends and classmates for their constant encouragement throughout the project.

Konda Vaishali - 2520030117

P. Bindhu Madhavi - 2520030077

Alaparthi Nava Harsha - 2520030493

ABSTRACT

The Smart Delivery Management System (branded **LogiFlow**) is a web-based application that simplifies, automates and improves the management of deliveries and shipments. It provides one platform for monitoring shipment details, tracking delivery status, reviewing transportation routes and maintaining delivery records. The main objectives are to reduce delivery delays, improve transportation efficiency and give logistics administrators clear visibility of every shipment.

The system is organised around a Dashboard, Shipment Management, Live Tracking with ETA, Route Optimization, Notifications, Delivery History, Database Analytics and a Distributed System view. From the database-engineering perspective, SQL operations (SELECT, INSERT, UPDATE, DELETE, JOIN and GROUP BY) manage shipment, customer and delivery records, while indexing techniques - B-Tree, Composite and Covering indexes - improve retrieval speed. Representative measurements shown in the prototype are 8 ms for Tracking ID search, 12 ms for Customer search and 21 ms for Delivery History.

From the distributed-systems perspective, the project applies database replication, read scaling, sharding and the CAP theorem. A primary database in Hyderabad replicates to read replicas in Bengaluru and Mumbai to support scalability, availability and reliable data management across locations.

The supplied implementation is a single-page HTML, CSS and JavaScript web interface with client-side navigation and demonstration data. Python (Flask) is the intended backend framework. This report documents the working interface and develops the database and distributed design for the full system without presenting proposed components (live GPS, authentication, real replication) as completed features.

Keywords: smart delivery, logistics, shipment tracking, SQL, indexing, replication, sharding, CAP theorem, Flask.

INDEX

CHAPTER 1 INTRODUCTION 8

1.1 Background of the Project 8

1.2 Problem Statement 8

1.3 Scope of the Project 9

1.4 Objectives 9

1.5 Importance of the Application 9

1.6 Target Users and Audience 10

CHAPTER 2 SYSTEM REQUIREMENTS 11

2.1 Hardware Requirements 11

2.2 Software Requirements 11

2.3 Development Tools and Frameworks 11

CHAPTER 3 TECHNOLOGY STACK 12

3.1 Front-end 12

3.2 Back-end 12

3.3 Database 12

3.4 Version Control 12

3.5 APIs and External Services 12

CHAPTER 4 SYSTEM ARCHITECTURE 13

4.1 High-level Architecture Diagram 13

4.2 Description of Each Layer 13

4.3 Deployment Architecture 14

CHAPTER 5 DESIGN 15

5.1 Interface and Workflow Design 15

5.2 ER Diagram and Database Schema 15

5.3 Normalization, Integrity and Indexing 17

CHAPTER 6 IMPLEMENTATION 19

6.1 Module-wise Implementation 19

6.2 Front-end Logic and State Management 19

6.3 API Integration 19

6.4 Authentication and Authorization 20

CHAPTER 7 FEATURES 21

7.1 List of Main Features 21

7.2 How Each Feature Works 21

CHAPTER 8 TESTING 23

8.1 Front-end and Backend Testing 23

8.2 Integration Testing 23

8.3 Tools Used for Testing 23

8.4 Test Cases and Results 24

CHAPTER 9 DEPLOYMENT 25

9.1 Steps to Deploy 25

9.2 Environment Configuration 25

9.3 Hosting of Frontend and Backend 25

CHAPTER 10 CHALLENGES AND LIMITATIONS 26

10.1 Issues Identified 26

10.2 Solutions Applied and Recommended 26

10.3 Current Limitations or Known Risks 26

CHAPTER 11 FUTURE ENHANCEMENTS 27

11.1 Planned Features 27

11.2 Possible Integrations and Optimizations 27

CHAPTER 12 CONCLUSION 28

12.1 Summary of the Project 28

12.2 What Was Achieved 28

12.3 Learning Outcomes 28

CHAPTER 13 REFERENCES 29

CHAPTER 14 APPENDICES 30

Appendix A Application Screenshots 30

Appendix B Sample Code and SQL 31

Appendix C Installation and User Guide 32

Appendix D Geo Tagged Photographs with Guide 32

Appendix E Review Form and Guide Signatures 33

CHAPTER 1 INTRODUCTION

1.1 Background of the Project

Delivery and logistics operations depend on timely knowledge of where each shipment is, who it belongs to, which route it is following and whether it is on schedule. When these details are handled manually - in registers, spreadsheets or separate messages - records become scattered, delays are noticed late and finding a single shipment becomes slow as the number of shipments and locations grows.

The Smart Delivery Management System addresses this by bringing shipment records, tracking, routes, notifications and delivery history into one web interface. Every shipment carries a unique tracking ID (for example SHP1001), the customer, the route (origin → destination), the package type, the delivery status, a priority and an estimated time of arrival (ETA).

Behind the interface sits a database layer. Shipments, customers and delivery records are related through stable identifiers, so a shipment can always be linked to its customer and its delivery history. Reusable information such as customer details is stored once and referenced, which reduces duplication and keeps records consistent.

Because delivery platforms are read-heavy - many people look up tracking status for every update that is written - the project also studies how indexing speeds up lookups and how a distributed design (a primary database with read replicas) lets the system scale across multiple cities.

The project therefore rests on three observations:

1. Manual handling of deliveries leads to late shipments, scattered records and slow lookups.
2. Administrators need a single, always-current view of every shipment and its status.
3. Tracking and history queries are frequent and must stay fast as data grows, which requires good indexing and a scalable distributed database design.

1.2 Problem Statement

Manual delivery handling causes late shipments, scattered records and slow lookups as operations grow across locations. Delayed deliveries are not identified quickly, the latest status of a shipment is hard to locate, route planning ignores distance, travel time and fuel cost, and database performance suffers when tracking searches and history queries are frequent. In a multi-location setup, the data layer must also consider replication, read scaling and partition tolerance.

The problem is therefore to design and develop an integrated delivery management application that centralises shipment data, enables fast tracking by ID, highlights delays, supports route planning information and maintains delivery history - backed by an indexed

relational schema and a distributed database architecture.

1.3 Scope of the Project

The project covers:

- **Dashboard:** live counts of total, in-transit, delayed and delivered shipments, a live delivery overview and a delivery-performance summary.
- **Shipment management:** a table to create, search and monitor shipments by tracking ID, customer or location.
- **Live tracking and ETA:** enter a tracking ID to see current route, status and ETA.
- **Route optimization:** pickup, destination, vehicle, distance, estimated time and fuel cost.
- **Operations:** notifications for new shipments, delays, successful deliveries and active transit, plus delivery history.
- **Database analytics:** SQL operations, query-performance indicators and the indexing strategy.
- **Distributed system view:** primary database, replicas, replication, read scaling, sharding and the CAP theorem.

Out of scope for the supplied prototype (listed as future work): a production Flask/REST backend connected to a persistent database, real-time GPS/map integration, authenticated users with role-based access and real database execution with monitoring.

1.4 Objectives

1. To provide a clear dashboard for monitoring shipment activity and delivery performance.
2. To enable shipment search and tracking using unique tracking IDs.
3. To support route-planning information such as distance, time and fuel cost.
4. To maintain a clear history of completed deliveries and operational notifications.
5. To demonstrate SQL operations, indexing and query-performance concepts.
6. To visualise distributed data infrastructure using replication, read scaling and sharding.
7. To reduce delivery delays, improve transportation efficiency and give full shipment visibility.

1.5 Importance of the Application

- **Fewer delays:** delayed shipments are flagged on the dashboard and through notifications so they can be acted on early.
- **Efficiency:** route information (distance, time, fuel cost) supports better transportation decisions.

- **Visibility:** every shipment can be found and tracked by a single ID.
- **Reliable records:** structured storage replaces scattered records and keeps delivery history accurate.
- **Performance at scale:** indexes keep common lookups fast as the data grows.
- **Availability:** replication across Hyderabad, Bengaluru and Mumbai lets reads be served close to users and keeps data available if a node fails.
- **Academic value:** the project connects SQL, indexing, replication, sharding and the CAP theorem to one realistic use case.

1.6 Target Users and Audience

User	How they use the system
Logistics administrators / managers	Monitor all shipments, identify delays, review analytics
Dispatch / operations staff	Create and update shipments, plan routes, review notifications
Customers (future scope)	Track their own shipment with a tracking ID
Students and faculty	Study indexing, SQL and distributed database concepts through a working example

CHAPTER 2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Component	Minimum	Recommended
Processor	Dual-core, 2 GHz	Quad-core, 2.5 GHz or higher
RAM	4 GB	8 GB
Storage	500 MB free	2 GB free (database and logs)
Display	1366 × 768	1920 × 1080
Network	Not required for the standalone prototype	Broadband for the Flask server and replicas

2.2 Software Requirements

Software	Purpose
Operating system	Windows 10/11, Linux or macOS
Web browser	Google Chrome or Microsoft Edge (latest) to run and test the interface
Python 3.x with Flask	Backend framework for server-side operations
SQL database (SQLite for development; MySQL/PostgreSQL for deployment)	Storage of shipments, customers and delivery records
Git / GitHub	Version control

2.3 Development Tools and Frameworks

- **Visual Studio Code** - editor for HTML, CSS, JavaScript and Python.
- **HTML5 / CSS3 / JavaScript** - interface structure, styling and interactivity.
- **Python (Flask)** - backend request handling and database connectivity.
- **SQL / DBMS** - schema, queries and indexes.
- **Google Chrome** - running, testing and debugging the web interface.

CHAPTER 3 TECHNOLOGY STACK

3.1 Front-end

The front-end is a responsive single-page application with a fixed navigation sidebar. **HTML5** provides the structure of the dashboard, shipment table and tracking page. **CSS3** provides the styling - cards, coloured status badges, tables, buttons and responsive layout. **JavaScript** provides client-side navigation between sections, rendering of shipment data, search and tracking interactions, and display of delivery status.

3.2 Back-end

Python (Flask) is the intended back-end framework. It is lightweight and suited to exposing REST endpoints such as shipment creation, tracking lookup and analytics. Flask processes requests, applies validation and application logic, and connects the web application to the database. In the supplied prototype the data is demonstration data held in the page, so Flask integration is part of the proposed full system.

3.3 Database

SQL and a **Database Management System (DBMS)** store and manage shipment information, customer details, delivery records and tracking data. SQLite is used for development; a server database such as MySQL or PostgreSQL is suitable for deployment with replication. The SQL operations used are SELECT, INSERT, UPDATE, DELETE, JOIN and GROUP BY. Indexing uses B-Tree, Composite and Covering indexes (see Chapter 5).

3.4 Version Control

Git and GitHub are used to track changes, collaborate among the three team members and keep a history of the project files.

3.5 APIs and External Services

The prototype uses no external services. Future versions can integrate a maps/GPS service for live location, an SMS/e-mail gateway for notifications and a cloud platform for hosting.

CHAPTER 4 SYSTEM ARCHITECTURE

4.1 High-level Architecture Diagram

Figure 4.1 shows the architecture of the proposed full system. The supplied prototype is a single browser page (HTML + CSS + JavaScript) with client-side navigation and demonstration data held in JavaScript arrays.

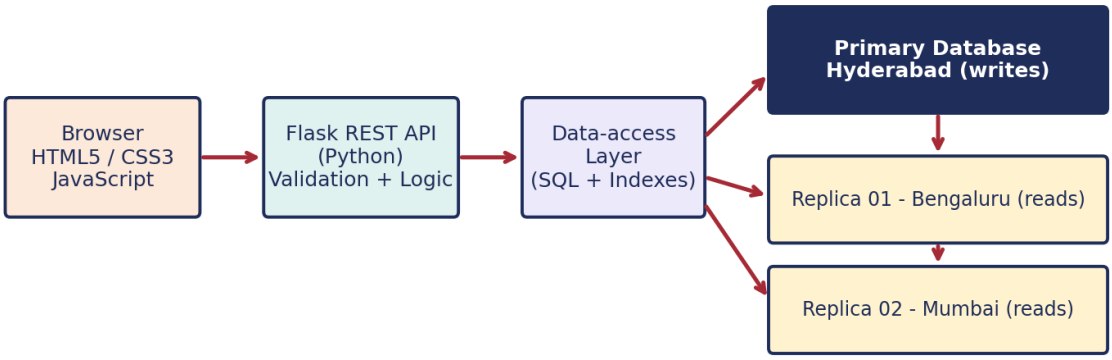


Figure 4.1 Architecture of the Smart Delivery Management System

4.2 Description of Each Layer

Layer	Responsibility
Presentation layer	Dashboard, shipment table, tracking page, route form, notifications, analytics and distributed-system views
Validation layer	Checks tracking ID format, required fields, valid status values and priority before data is saved
Application (Flask) layer	Receives requests, applies business rules (for example marking a shipment Delayed when ETA passes), builds responses
Data-access layer	Executes parameterised SQL queries and transactions
Persistence layer	Relational database holding customers, shipments, routes, history and notifications
Distribution layer	Replication from primary to replicas, read routing and sharding

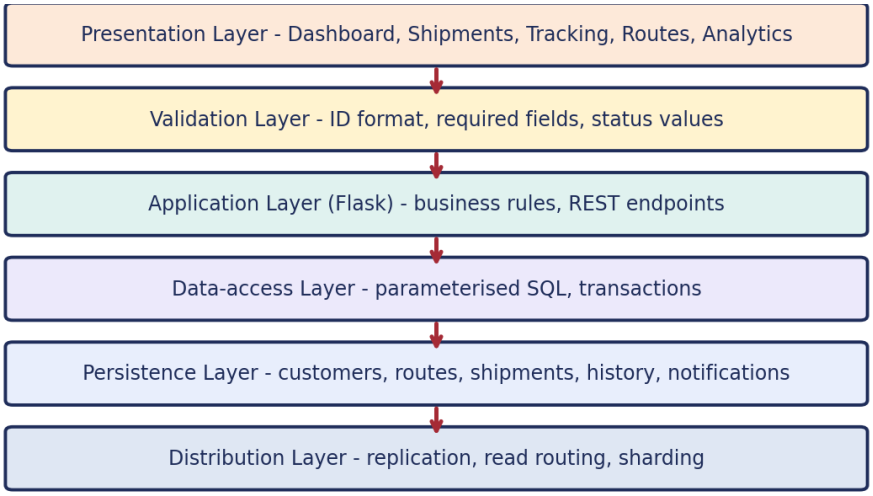


Figure 4.2 Layered architecture

4.3 Deployment Architecture

The distributed deployment uses a **primary-replica** model:

- **Primary Database - Hyderabad:** accepts all writes (new shipments, status updates).
- **Replica 01 - Bengaluru** and **Replica 02 - Mumbai:** receive changes from the primary through replication and serve read queries such as tracking lookups.

Replication: each committed change on the primary is copied to the replicas, giving redundancy and a backup if a node fails.

Read scaling: because tracking and history queries are read-heavy, they are routed to the nearest replica, reducing load on the primary and latency for users.

Sharding: as data grows, shipments can be partitioned horizontally, for example by region or by a hash of the tracking ID, so that no single database holds all rows.

CAP theorem: a distributed database can guarantee only two of Consistency, Availability and Partition tolerance during a network partition. For logistics, tracking reads favour **availability** (showing the latest known status) with eventual consistency on replicas, while writes on the primary remain strongly consistent.

CHAPTER 5 DESIGN

5.1 Interface and Workflow Design

The interface uses a fixed left sidebar with these sections: Dashboard, Shipments, Live Tracking, Route Optimization, Notifications, Delivery History, Database Analytics and Distributed System. A top bar shows notifications and the logged-in administrator.

Typical workflow:

1. The administrator opens the Dashboard and reads the counts of total, in-transit, delayed and delivered shipments.
2. In Shipments, a new shipment is created with customer, route, package type, priority and ETA.
3. The shipment is monitored in the shipment table; status badges (Picked Up, In Transit, Out for Delivery, Delayed, Delivered) show progress.
4. A tracking ID entered in Live Tracking returns the route, status and ETA immediately.
5. Delays and deliveries create notifications; completed deliveries move to Delivery History.
6. Database Analytics and Distributed System pages explain how the data layer supports these actions.

Demonstration data: the dashboard shows six sample shipments, for example SHP1001 (Rahul Kumar, Hyderabad → Bengaluru, Electronics, In Transit), SHP1004 (Sneha Rao, Chennai → Bengaluru, Medical Supplies, Delayed, High priority) and SHP1003 (Arun Reddy, Mumbai → Pune, Clothing, Delivered).

5.2 ER Diagram and Database Schema

The proposed relational model has the following one-to-many relationships:

- One **Customer** places many **Shipments**.
- One **Route** is used by many **Shipments**.
- One **Shipment** has many **Delivery History** (status update) records and many **Notifications**.

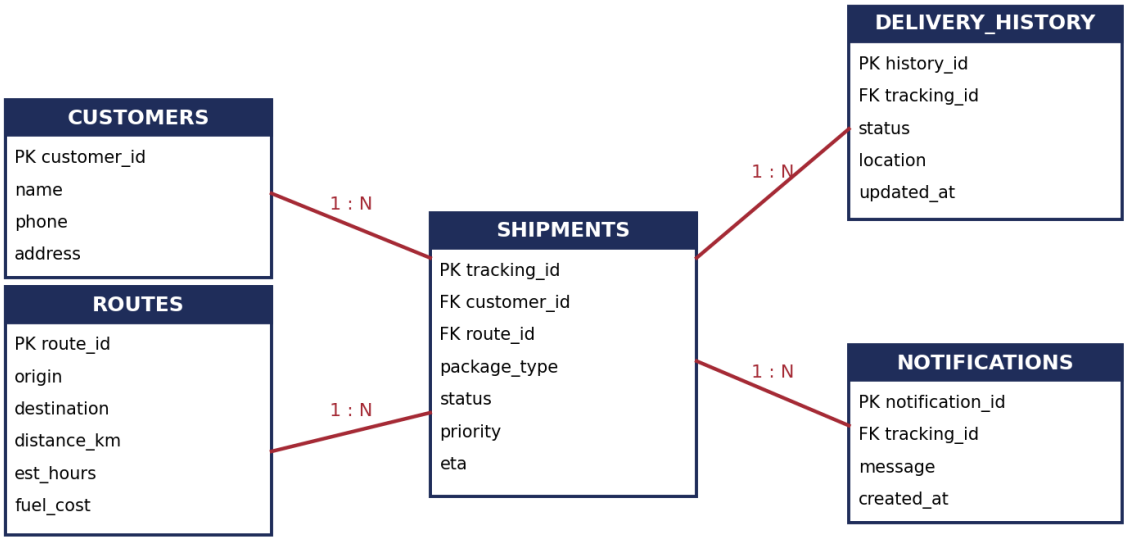


Figure 5.1 Proposed ER model with one-to-many relationships

Table: customers

Column	Type	Constraint
customer_id	INTEGER	Primary key
name	VARCHAR(80)	NOT NULL
phone	VARCHAR(15)	
address	VARCHAR(200)	

Table: routes

Column	Type	Constraint
route_id	INTEGER	Primary key
origin	VARCHAR(60)	NOT NULL
destination	VARCHAR(60)	NOT NULL
distance_km	DECIMAL(8,2)	
est_hours	DECIMAL(5,2)	
fuel_cost	DECIMAL(10,2)	

Table: shipments

Column	Type	Constraint
tracking_id	VARCHAR(12)	Primary key (e.g. SHP1001)
customer_id	INTEGER	Foreign key → customers
route_id	INTEGER	Foreign key → routes
package_type	VARCHAR(40)	
status	VARCHAR(20)	CHECK in (Picked Up, In Transit, Out for Delivery, Delayed, Delivered)
priority	VARCHAR(10)	CHECK in (Normal, High)
eta	DATETIME	
created_at	DATETIME	DEFAULT current time

Table: delivery_history

Column	Type	Constraint
history_id	INTEGER	Primary key
tracking_id	VARCHAR(12)	Foreign key → shipments
status	VARCHAR(20)	NOT NULL
location	VARCHAR(60)	
updated_at	DATETIME	NOT NULL

Table: notifications

Column	Type	Constraint
notification_id	INTEGER	Primary key
tracking_id	VARCHAR(12)	Foreign key → shipments
message	VARCHAR(200)	
created_at	DATETIME	

5.3 Normalization, Integrity and Indexing

Normalization: customer details are stored once in customers and route details once in routes; shipments only reference them by key. All tables are in Third Normal Form (3NF) - each non-key column depends on the key, the whole key and nothing but the key - which removes update, insert and delete anomalies.

Integrity: primary keys identify rows, foreign keys keep references valid, CHECK constraints restrict status and priority values, and transactions keep related updates

(shipment status plus history row) atomic.

Indexing strategy:

Index	Type	Purpose
shipments(tracking_id)	B-Tree (primary key)	Fast Tracking ID search
customers(name)	B-Tree	Customer search
delivery_history(tracking_id, updated_at)	Composite	Delivery history in time order
shipments(status, priority, eta)	Covering	Dashboard counts and delayed-shipment lists answered from the index alone

Representative query performance (shown in the prototype):

Query	Time
Tracking ID search	8 ms
Customer search	12 ms
Delivery history	21 ms

These figures are demonstration indicators of the benefit of indexing, not benchmarked measurements from a production database.

CHAPTER 6 IMPLEMENTATION

6.1 Module-wise Implementation

No.	Module	Description
1	Dashboard	Shipment statistics, live delivery overview and recent activity
2	Shipment Management	Shipment table with search by tracking ID, customer or location
3	Live Tracking	Tracking-ID lookup showing current status and ETA
4	Route Optimization	Pickup, destination, vehicle, distance, time and fuel cost
5	Operations	Notifications and delivery history
6	Database Analytics	SQL operations, query performance and indexing strategy
7	Distributed System	Primary database, replicas, replication, read scaling and sharding

6.2 Front-end Logic and State Management

The page keeps the shipment list as a JavaScript array. A navigation function shows one section and hides the others without reloading the page. Rendering functions build the shipment table rows and status badges from the array. The search box filters the array by tracking ID, customer or location as the user types, and the tracking form looks up a tracking ID and displays route, status and ETA (for example SHP1001: Hyderabad → Bengaluru, In Transit, ETA today 2:30 PM). Dashboard counters are computed from the same array so they always agree with the table.

6.3 API Integration

The prototype has no live API. In the full system, Flask exposes REST endpoints:

Method	Endpoint	Purpose
GET	/api/shipments	List and search shipments
POST	/api/shipments	Create a shipment (INSERT)
PUT	/api/shipments/{tracking_id}	Update status (UPDATE)
DELETE	/api/shipments/{tracking_id}	Remove a shipment (DELETE)
GET	/api/track/{tracking_id}	Live tracking and ETA
GET	/api/analytics/summary	Counts by status (GROUP BY)
GET	/api/history/{tracking_id}	Delivery history (JOIN)

6.4 Authentication and Authorization

The prototype shows an "Admin / Manager" profile but does not authenticate. The planned design uses password-hashed logins with session or token authentication and role-based access: administrators manage all shipments, staff create and update shipments, and customers (future) may only track their own shipments.

CHAPTER 7 FEATURES

7.1 List of Main Features

1. Live Dashboard
2. Shipment Management
3. Live Tracking and ETA
4. Smart Notifications
5. Query Analytics (SQL operations, indexing, performance)
6. Route Optimization information
7. Delivery History
8. Distributed System view

7.2 How Each Feature Works

Live Dashboard: counts total, in-transit, delayed and delivered shipments and shows a live delivery overview and delivery-performance bars.

Shipment Management: lists every shipment with tracking ID, customer, route, package, status, priority and ETA; supports creation and search.

Live Tracking and ETA: the user enters a tracking ID and immediately sees the route, status and ETA.

Smart Notifications: highlights new shipments, delays, successful deliveries and active transit.

Query Analytics: presents SELECT, INSERT, UPDATE, DELETE, JOIN and GROUP BY usage and the query-time indicators.

Route Optimization: shows distance, estimated travel time and fuel cost for pickup-destination pairs.

Distributed System: shows the Hyderabad primary with Bengaluru and Mumbai replicas, and explains replication, read scaling, sharding and CAP.

Sample SQL used by the features:

```
-- Live tracking (B-Tree lookup)
SELECT s.tracking_id, r.origin, r.destination, s.status, s.eta
FROM shipments s JOIN routes r ON s.route_id = r.route_id
WHERE s.tracking_id = 'SHP1001';

-- Dashboard counts
SELECT status, COUNT(*) AS total FROM shipments GROUP BY status;

-- Create shipment
INSERT INTO shipments (tracking_id, customer_id, route_id,
                       package_type, status, priority, eta)
VALUES ('SHP1007', 3, 2, 'Electronics', 'Picked Up', 'Normal',
        '2026-11-02 18:00');

-- Update status
UPDATE shipments SET status = 'Delivered' WHERE tracking_id = 'SHP1003';

-- Delivery history
SELECT h.status, h.location, h.updated_at
FROM delivery_history h
WHERE h.tracking_id = 'SHP1001' ORDER BY h.updated_at;
```

CHAPTER 8 TESTING

8.1 Front-end and Backend Testing

Front-end testing checks that every sidebar section opens, the shipment table renders all rows with correct status badges, search filters correctly, the tracking form returns the right result and the dashboard counters match the table. The layout was checked in Google Chrome at desktop and reduced widths.

Backend and database testing (for the full system) checks that INSERT, UPDATE, DELETE and SELECT statements behave correctly, that foreign-key and CHECK constraints reject invalid data, that JOIN and GROUP BY queries return correct results and that indexed queries use the intended index (verified with EXPLAIN).

8.2 Integration Testing

Integration testing verifies that the layers work together: a shipment created in the interface appears in the table and in dashboard counts; updating a status to Delayed raises a notification; a tracking ID typed in Live Tracking returns the same status shown in the shipment table; a delivered shipment appears in Delivery History. In the full system, the same flow is tested through the Flask API and database, and read queries are checked against replicas after a write on the primary.

8.3 Tools Used for Testing

Tool	Use
Google Chrome DevTools	Console errors, layout and responsive checks
Manual test cases	Functional verification of each module
Postman / curl	API endpoint testing (full system)
SQL EXPLAIN	Confirming index usage

8.4 Test Cases and Results

ID	Test case	Expected result	Result
TC-01	Open Dashboard	Counts of total, in-transit, delayed and delivered shipments shown	Pass
TC-02	Open Shipments page	All shipments listed with status, priority and ETA	Pass
TC-03	Search by customer name	Only matching shipments displayed	Pass
TC-04	Track SHP1001	Hyderabad → Bengaluru, In Transit, ETA shown	Pass
TC-05	Track an unknown ID	A "not found" style message, no error	Pass
TC-06	Delayed shipment	Shown with Delayed badge and High priority	Pass
TC-07	Open Notifications	New, delayed and delivered events listed	Pass
TC-08	Open Database Analytics	SQL operations and query times displayed	Pass
TC-09	Open Distributed System	Primary and two replicas displayed	Pass
TC-10	Backend INSERT/UPDATE/DELETE (full system)	Row changes persisted	To be tested after Flask integration

TC-01 to TC-09 apply to the supplied prototype; TC-10 applies to the proposed backend.

CHAPTER 9 DEPLOYMENT

9.1 Steps to Deploy

Prototype: copy the HTML file to any folder and open it in Chrome or Edge, or host it as a static page.

Full system:

1. Install Python 3 and create a virtual environment.
2. Install Flask and the database driver (pip install flask).
3. Create the database from the schema script and load sample data.
4. Create the indexes listed in Section 5.3.
5. Run the Flask application (flask run) and open the interface in the browser.
6. Configure replicas and point read queries to them.

9.2 Environment Configuration

Setting	Purpose
DATABASE_URL	Primary database connection (writes)
REPLICA_URLS	Replica connections (reads)
SECRET_KEY	Session/token signing
FLASK_ENV	development or production

Secrets are kept in environment variables, never in source code.

9.3 Hosting of Frontend and Backend

The front-end can be served as static files (GitHub Pages, Netlify or the Flask static folder). The Flask back-end can be hosted on a cloud VM or platform service behind a production web server. The primary database is placed in Hyderabad with replicas in Bengaluru and Mumbai, matching the distributed architecture in Chapter 4.

CHAPTER 10 CHALLENGES AND LIMITATIONS

10.1 Issues Identified

- Tracking data is spread across several views and must stay consistent.
- Lookups on large tables become slow without indexes.
- Replicas can briefly show older data than the primary.
- Route planning must balance distance, time, fuel cost and delays.

10.2 Solutions Applied and Recommended

Issue	Solution
Scattered data	Single shipment array/table is the source for dashboard, table and tracking
Slow lookups	B-Tree, Composite and Covering indexes
Replica lag	Accept eventual consistency for reads; read from primary when exact status is critical
Growth of data	Sharding by region or tracking-ID hash
Route decisions	Show distance, time and fuel cost together for comparison

10.3 Current Limitations or Known Risks

- The prototype uses demonstration data and is not connected to a persistent database.
- Flask backend, authentication and role-based access are not yet implemented.
- Live GPS and map tracking are not available.
- Query times and the replication diagram are illustrative, not measured on a live replicated cluster.
- Route optimization displays route information; it does not compute optimal routes with an algorithm.

CHAPTER 11 FUTURE ENHANCEMENTS

11.1 Planned Features

- Production Flask/REST backend with a persistent database.
- Real-time GPS and map integration for live location and accurate ETA.
- Authenticated users and role-based administration.
- Customer portal for self-service tracking.
- SMS/e-mail notifications for status changes and delays.
- Algorithmic route optimization (shortest path / delivery sequencing).

11.2 Possible Integrations and Optimizations

- Real database execution with monitoring dashboards for query performance.
- Cloud deployment with managed replicas and automatic failover.
- Caching of frequent tracking lookups.
- Analytics on delay causes and delivery performance by region.
- Mobile app for drivers to update status on the road.

CHAPTER 12 CONCLUSION

12.1 Summary of the Project

The Smart Delivery Management System brings shipment monitoring, tracking, route planning, notifications and delivery history into one web interface. The database analytics view demonstrates SQL operations, indexing and query-performance concepts, and the distributed-system view explains how a primary database with replicas, read scaling and sharding can support a larger logistics platform.

12.2 What Was Achieved

- A responsive single-page interface with eight functional sections.
- A shipment model with tracking ID, customer, route, package, status, priority and ETA.
- Live tracking by ID with status and ETA.
- A normalised, indexed relational schema design.
- A distributed architecture design using replication, read scaling, sharding and CAP reasoning.

12.3 Learning Outcomes

- Designing a normalised relational schema and writing SELECT, INSERT, UPDATE, DELETE, JOIN and GROUP BY queries.
- Choosing B-Tree, Composite and Covering indexes for real query patterns.
- Understanding replication, read scaling, sharding and the CAP trade-offs.
- Building a responsive front-end with HTML5, CSS3 and JavaScript and planning a Flask back-end.
- Teamwork and version control with Git and GitHub.

CHAPTER 13 REFERENCES

1. Silberschatz, A., Korth, H. F., Sudarshan, S., Database System Concepts, McGraw-Hill.
2. Elmasri, R., Navathe, S. B., Fundamentals of Database Systems, Pearson.
3. Kleppmann, M., Designing Data-Intensive Applications, O'Reilly Media.
4. Flask Documentation - https://flask.palletsprojects.com
5. SQLite Documentation - https://www.sqlite.org/docs.html
6. MDN Web Docs (HTML, CSS, JavaScript) - https://developer.mozilla.org
7. Gilbert, S., Lynch, N., "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services", ACM SIGACT News, 2002.
8. Course material, Database Systems Engineering and Distributed Backend Development (25CS1302E), K L University.

CHAPTER 14 APPENDICES

Appendix A Application Screenshots

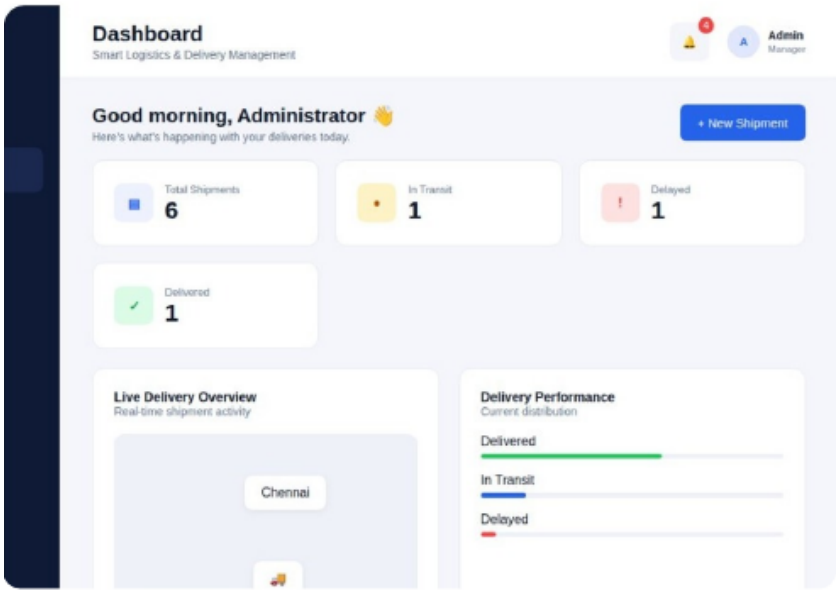


Figure A.1 Dashboard

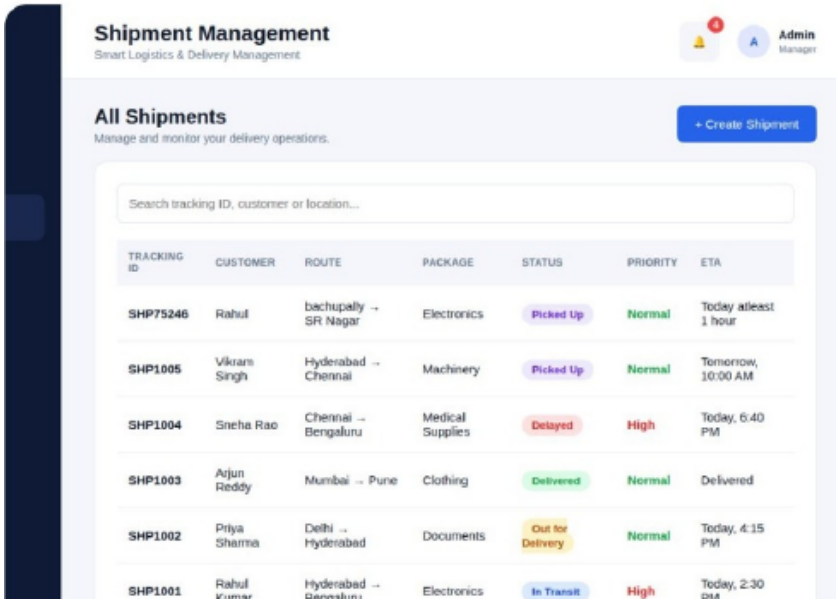


Figure A.2 Shipment Management

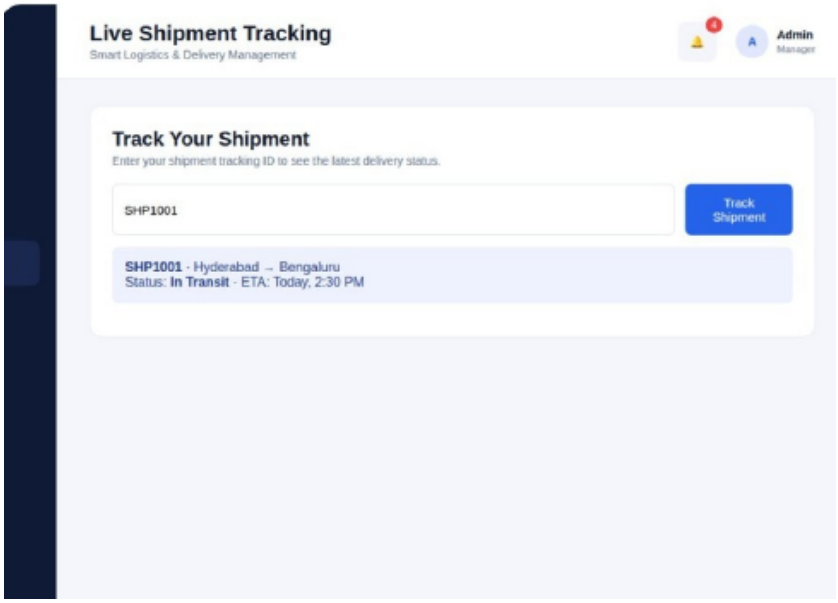


Figure A.3 Live Shipment Tracking

Appendix B Sample Code and SQL

```
CREATE TABLE customers (
  customer_id INTEGER PRIMARY KEY,
  name VARCHAR(80) NOT NULL,
  phone VARCHAR(15),
  address VARCHAR(200)
);

CREATE TABLE routes (
  route_id INTEGER PRIMARY KEY,
  origin VARCHAR(60) NOT NULL,
  destination VARCHAR(60) NOT NULL,
  distance_km DECIMAL(8,2),
  est_hours DECIMAL(5,2),
  fuel_cost DECIMAL(10,2)
);

CREATE TABLE shipments (
  tracking_id VARCHAR(12) PRIMARY KEY,
  customer_id INTEGER REFERENCES customers(customer_id),
  route_id INTEGER REFERENCES routes(route_id),
  package_type VARCHAR(40),
  status VARCHAR(20) CHECK (status IN ('Picked Up','In Transit',
         'Out for Delivery','Delayed','Delivered')),
  priority VARCHAR(10) CHECK (priority IN ('Normal','High')),
  eta DATETIME,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE delivery_history (
  history_id INTEGER PRIMARY KEY,
  tracking_id VARCHAR(12) REFERENCES shipments(tracking_id),
  status VARCHAR(20) NOT NULL,
  location VARCHAR(60),
  updated_at DATETIME NOT NULL
);

CREATE INDEX idx_customer_name ON customers(name);
CREATE INDEX idx_history ON delivery_history(tracking_id, updated_at);
CREATE INDEX idx_dashboard ON shipments(status, priority, eta);

# Flask tracking endpoint (proposed)
from flask import Flask, jsonify
```

```
import sqlite3

app = Flask(__name__)

@app.route("/api/track/<tracking_id>")
def track(tracking_id):
    db = sqlite3.connect("logiflow.db")
    row = db.execute(
        "SELECT s.tracking_id, r.origin, r.destination, s.status, s.eta "
        "FROM shipments s JOIN routes r ON s.route_id = r.route_id "
        "WHERE s.tracking_id = ?", (tracking_id,)).fetchone()
    if row is None:
        return jsonify(error="Tracking ID not found"), 404
    keys = ["tracking_id", "origin", "destination", "status", "eta"]
    return jsonify(dict(zip(keys, row)))
```

Appendix C Installation and User Guide

1. Download the project folder and open it in Visual Studio Code.
2. Open the HTML file in Google Chrome (or run the Flask server and open the local address).
3. Use the sidebar to move between Dashboard, Shipments, Live Tracking, Route Optimization, Notifications, Delivery History, Database Analytics and Distributed System.
4. To track a parcel, open Live Tracking, type a tracking ID such as SHP1001 and press Track Shipment.
5. To find a shipment, type a tracking ID, customer name or location in the search box on the Shipments page.
6. To add a shipment, click Create Shipment and fill in the details.

Appendix D Geo Tagged Photographs with Guide

Insert geo-tagged photographs taken with the guide, Dr. Spandana. S.

Appendix E Review Form and Guide Signatures

Review	Date	Remarks	Guide signature
Review 1			
Review 2			
Final review			